

Overall Approach to Implementation

Our implementation was centred around an agile style of programming, as we were more focused on function increments than refining software quality. We started with the core entity classes and map structure before integrating game logic and the graphical interface. We divided the codebase into distinct components and assigned each team member the responsibility of a specific area, allowing parallel development with minimal overlap. All source code was organized following the standard Maven project layout, with production classes maintained under **src/main/java**. Integration of components was done progressively as individual parts reached a functional state. In addition to individual contributions, the team held regular working sessions throughout the week where all members collaborated simultaneously, allowing for real-time problem solving and faster integration of components.

Milestones

Milestone 1 — Halfway Progress (March 5th)

By the halfway deadline, each team member pushed an initial version of their assigned components: **Luke** handled map creation and tile rendering, **Armin** implemented the player and enemy classes (dynamic entities), **Ian** built the reward/punishment system and the main entity superclass, and **Parsa** set up the GUI and interface using Java Swing. These foundational components established the basic structure of the game and ensured all core classes were in place before integration work began.

Milestone 2 — Final Submission (March 16th)

The final milestone focuses on full implementation of all files, including user input, map creation using proper asserts, scoring, entity spawning, and enemy behaviour and movement. This milestone also includes completing the Phase 2 report with a full account of UML and design changes, management process, challenges, and external references, as well as outlining goals for Phases 3 and 4.

Adjustments and Modifications from Phase 1

Several adjustments were made to our Phase 1 design as implementation revealed areas where the original design was insufficient or inaccurate.

- The **Tile_Class** enum was renamed to **Tile_Type** for clarity and consistency with naming conventions used across the codebase.
- The score attribute in the **Game** class was made public to allow direct access from other components that needed to read and update it during gameplay.

- **Speed** was refactored from a fixed value into a dynamic, mutable variable to support gameplay mechanics. Getters and setters were added accordingly.
- A new **GamePanel** class was added to manage the game window, which was not accounted for in the original design but proved necessary once GUI implementation began.
- We added a **Map_Builder** class to actually build the map and have a place where we can easily change the map. This was done by having a String map, which was input into a map of Map tiles. This allowed us to just type a character to create a change in the map. i.e. 'wwfffsbsbfff'. Changing any of these characters changes the map, allowing for easy map design.
- Added a draw function to each map class to draw the components of the map onto the game panel.
- Also, we added necessary methods in gameMap, such as getCols(), get(Rows), and methods for boundary checking to prevent bugs from occurring.
- We added lots of attributes and methods to the new **GamePanel** class for game functionality, such as movement and checking for game conditions.
- Overall, the primary theme of our Phase 1 adjustments was underestimating the number of classes and files required, as many components only became apparent once implementation began. We added additional helper classes like **GameStateResetHelper** and **PlayerMovementHelper**, and this aided us in keeping GamePanel from growing excessively large.
- On the other hand, we had some classes that were at first introduced for the abstraction of the code, but then ended up being unnecessary during implementation, which led to us removing them. This reflects the areas where our initial pattern design was wrong and overestimated the complexity of certain parts of the assignment.

Division of Roles and Responsibilities

Work was divided by component, with each team member taking full ownership of their assigned area:

- Luke → Map creation and tile rendering
- Armin → Player and enemy implementation (dynamic entities), as well as sprite asset management
 - Including but not limited to:
 - Enemy Package
 - SecurityGuard
 - Player Class
 - Player Sprites
- Ian → Reward and punishment classes (stationary entities) and the main entity superclass

- Parsa → GUI and interface implementation using Java Swing, as well as PlayerMovement with Armin

This division allowed team members to work independently on separate files, reducing the likelihood of conflicts and making individual contributions easier to track through version history while also allowing everybody to contribute in equal amounts to the project.

External Libraries Used

- Java Swing Toolkit — Used for building the graphical user interface. Swing was chosen for its native integration with Java, requiring no additional dependencies and ensuring full compatibility with our Maven build setup. It provides sufficient functionality for rendering the game window, handling user input, and managing UI components without introducing unnecessary complexity.

Measures to Improve the Quality of Code

- JavaDoc comments were written for all classes and methods to document purpose, parameters, and return values, making the codebase easier to understand and maintain across the team.
- Team members were given different roles and responsibilities, which allowed for easier coordination between classes and for maintainable code for future proofing.
- The single-responsibility principle was applied when structuring classes — particularly when splitting larger components into smaller, focused files, ensuring each class has a clear and well-defined role.
- Consistent naming conventions were established and followed across all files to improve readability and reduce ambiguity when integrating each member's work.

Biggest Challenges During This Phase

- Coordinating parallel development across four contributors initially caused merge conflicts as team members occasionally worked on overlapping areas of the codebase. We addressed this through more active communication and by decomposing larger components into smaller, modular files, which allowed each team member to work on distinct classes with minimal overlap. Establishing these practices early made integration significantly smoother as the project progressed.
- One constant challenge that we faced was the tendency to centralize logic in a single file for convenience. It was difficult for us to fully leverage the object-oriented design, and getting the right balance between practicality and clean separation of code into different classes was something the team grew into over the course of the phase.

Goals for Phase 3

- Going forward, we will adhere more closely to object-oriented design, which will help us further reduce issues such as merge conflicts while ensuring a cleaner codebase and a better separation of responsibilities.
- Change our rewards from squares to sprites, in a grocery store theme (berries, meat, and chocolate).
- Have a main menu for the start of the game, which will add a more professional and put-together feel to the game.
- Make the maze design more unique, rather than just having rows of objects.
- Have the camera be closer to the player so the whole map isn't in frame.